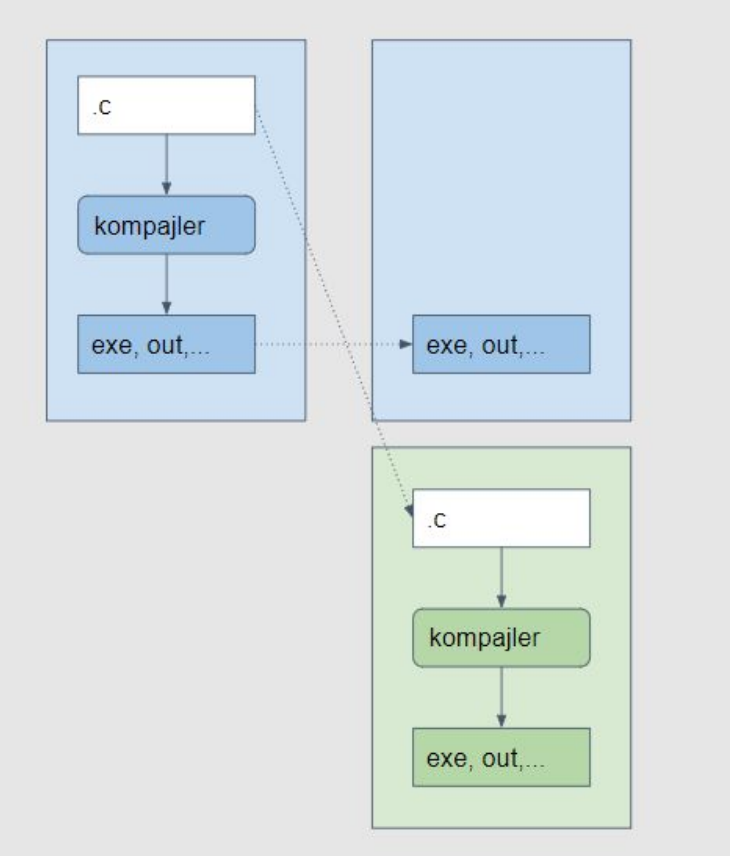
Programski prevodioci

12 python

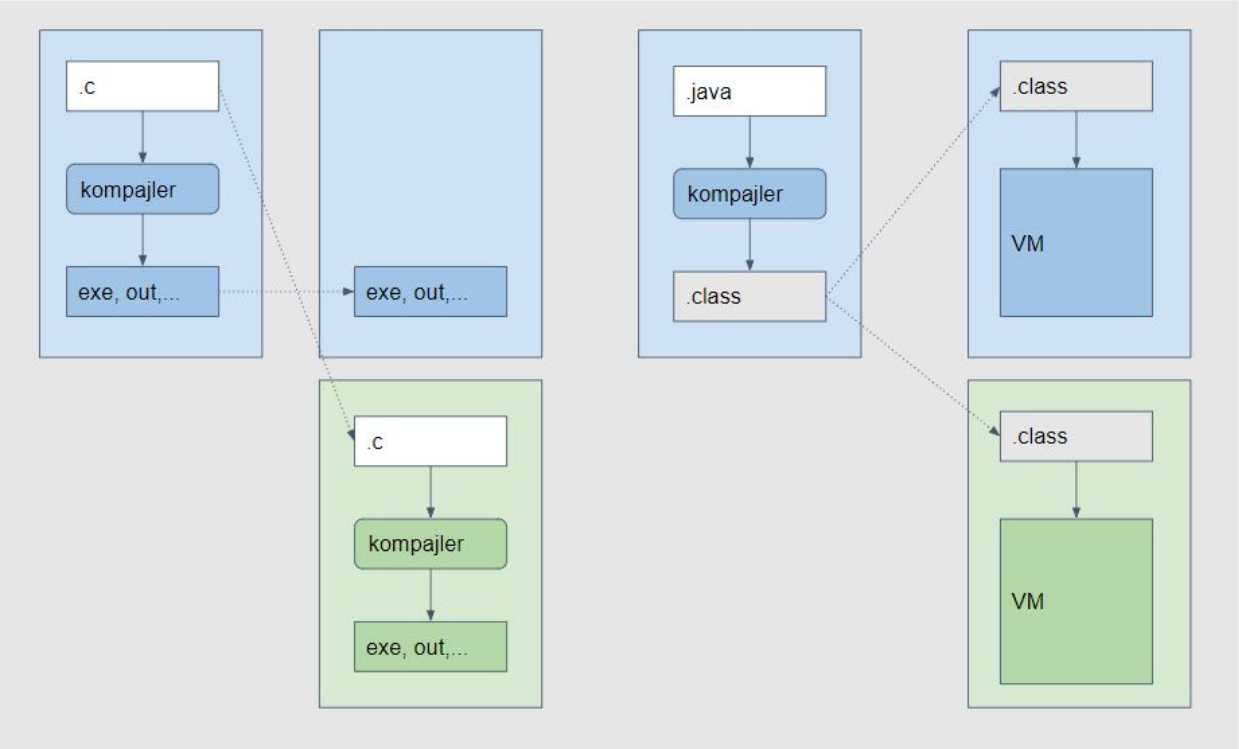
25-26

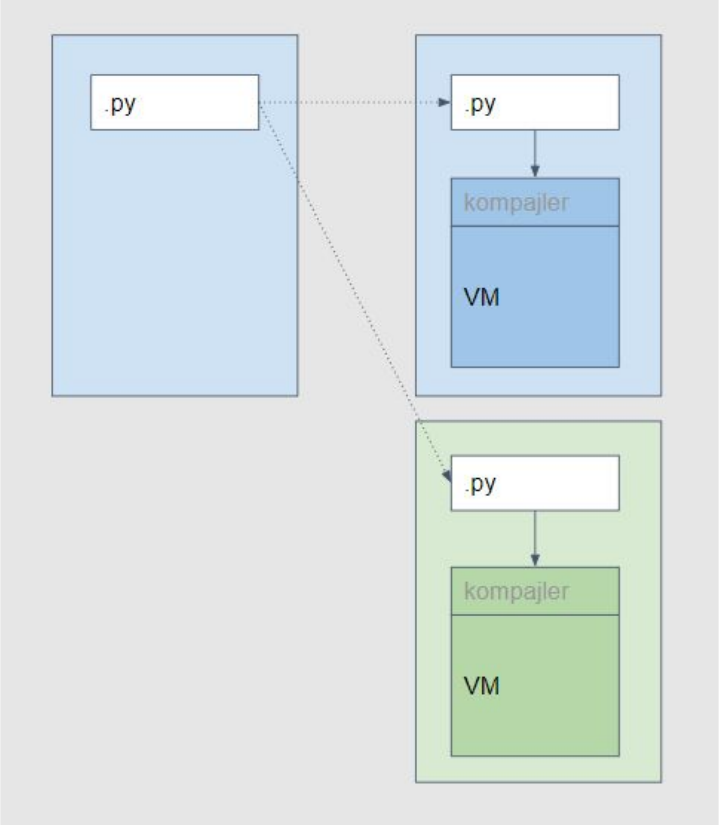
Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad



Ilustracija, sve je pojednostavljeno


.c
kompajler
exe, out,...
exe, out,...
.c
kompajler
exe, out,...
.java
kompajler
.class
.class
VM
.class
VM


.py
.py
kompajler
VM
.py
kompajler
VM

- Program se prevodi na bytecode (međukod)
- PVM izvršava taj bytecode (interpreter)

- Često korisnici nisu svesni da postoji prevođenje (kompajler) jer ga ne pozivamo eksplicitno i uglavom odmah vidimo rezultate izvršavanja.

- javac - kompajler - izlaz class fajlovi koji sadrže bytecode (nečitljiv)
- java - izvršava class na JVM (interpreter)
- javap - disasembler za class

- python - kompajler i PVM interpreter

- py_compile... - izlaz .pyc fajl koji sadrži bytecode (nečitljiv)
  `(python -m py_compile test.py // py_compile.compile(...))`

- dis - podrška za disasembler za bytecode
  `(python -m dis test.py // dis.dis(...))`

```
def saberi():
      a = 5
      b = 3
      a = a + b
      print(a)
```

```
  2           0 LOAD_CONST               1 (5)
              3 STORE_FAST               0 (a)

  3           6 LOAD_CONST               2 (3)
              9 STORE_FAST               1 (b)

  4          12 LOAD_FAST                0 (a)
             15 LOAD_FAST                1 (b)
             18 BINARY_ADD
             19 STORE_FAST               0 (a)

  5          22 LOAD_GLOBAL              0 (print)
             25 LOAD_FAST                0 (a)
             28 CALL_FUNCTION            1
             31 POP_TOP
             32 LOAD_CONST               0 (None)
             35 RETURN_VALUE
```

```
import test
import dis
dis.dis(test.saberi)
```

```python
def saberi():
    a = 5
    b = 3
    a = a + b
    print(a)
```

```
Name:              saberi
Filename:          ...\test.py
Argument count:    0
Kw-only arguments: 0
Number of locals:  2
Stack size:        2
Flags:             OPTIMIZED, NEWLOCALS, NOFREE
Constants:
   0: None
   1: 5
   2: 3
Names:
   0: print
Variable names:
   0: a
   1: b
```

```python
import test
import dis
dis.show_code(test.saberi)
```

python

Type / to searc

Overview | Repositories 82 | Projects 33 | Packages | People 123

Python

Repositories related to the Python Programming language

18.6k followers https://www.python.org/ Verified

Pinned

cpython Public
The Python programming language
Python 57.9k 28.7k

mypy Public
Optional static typing for Python
Python 17.1k 2.8k

pythondotorg Public
Source code for python.org
Python 1.4k 653

peps Public
Python Enhancement Proposals
reStructuredText 4k 1.6k

typeshed Public
Collection of library stubs for Python, with static types
Python 3.9k 1.7k

devguide Public
The Python developer's guide
Python 1.7k 747

Python Enhancement Proposals | Python » PEP Index » PEP 0

Contents

- Introduction
- Topics
- Index by Category
  - Meta-PEPs (PEPs about PEPs or Processes)
  - Other Informational PEPs
  - Provisional PEPs (provisionally accepted; interface may still change)
  - Accepted PEPs (accepted; may not be implemented yet)
  - Open PEPs (under consideration)
  - Finished PEPs (done, with a stable interface)
  - Historical Meta-PEPs and Informational PEPs
  - Deferred PEPs (postponed pending further research or updates)
  - Abandoned, Withdrawn, and Rejected PEPs
- Numerical Index
- Reserved PEP Numbers
- PEP Types Key
- PEP Status Key
- Authors/Owners

PEP 0 – Index of Python Enhancement Proposals (PEPs)

Author: The PEP Editors
Status: Active
Type: Informational
Created: 13-Jul-2000

▸ Table of Contents

Introduction

This PEP contains the index of all Python Enhancement Proposals, known as PEPs. PEP numbers are assigned by the PEP editors, and once assigned are never changed. The version control history of the PEP texts represent their historical record.

Topics

PEPs for specialist subjects are indexed by topic.

- Governance PEPs
- Packaging PEPs
- Release PEPs
- Typing PEPs

PEP 617 – New PEG parser for CPython

Author: Guido van Rossum <guido at python.org>, Pablo Galindo <pablogsal at python.org>, Lysandros Nikolaou <lisandrosnik at gmail.com>
Discussions-To: Python-Dev list
Status: Final
Type: Standards Track
Created: 24-Mar-2020
Python-Version: 3.9
Post-History: 02-Apr-2020

▶ **Table of Contents**

Important

This PEP is a historical document. The up-to-date, canonical documentation can now be found at Full Grammar specification.

See PEP 1 for how to propose changes.

Overview

This PEP proposes replacing the current LL(1)-based parser of CPython with a new PEG-based parser. This new parser would allow the elimination of multiple "hacks" that exist in the current grammar to circumvent the LL(1)-limitation. It would substantially reduce the maintenance costs in some areas related to the compiling pipeline such as the grammar, the parser and the AST generation. The new PEG parser will also lift the LL(1) restriction on the current Python grammar.

peps.python.org/pep-0617/

- Rationale
  - Some rules are not actually LL(1)
  - Complicated AST parsing
  - Lack of left recursion
  - Intermediate parse tree

LL(1) - Poslednje predavanje

Python Developer's Guide

Search

Getting started
Development workflow
Issues and triaging
Documentation
Testing and buildbots
Development tools
Core developers
CPython's internals
CPython source code

Guide to the parser

Abstract

The Parser in CPython is currently a PEG (Parser Expression Grammar) parser. The first version of the parser used to be an LL(1) based parser that was one of the oldest parts of CPython implemented before it was replaced by **PEP 617**. In particular, both the current parser and the old LL(1) parser are the output of a parser generator. This means that the way the parser is written is by feeding a description of the Grammar of the Python language to a special program (the parser generator) which outputs the parser. The way the Python language is changed is therefore by modifying the grammar file and developers rarely need to interact with the parser generator itself other than use it to generate the parser.

How PEG parsers work

A PEG (Parsing Expression Grammar) grammar (like the current one) differs from a context-free grammar in that the way it is written more closely reflects how the parser will operate when parsing it. The fundamental technical difference is that the choice operator is ordered. This means that when writing:

```
rule: A | B | C
```

devguide.python.org/internals/parser/

```
# PEG grammar for Python

...

simple_stmt:
    | assignment
    | star_expressions
    | return_stmt
    | import_stmt
...
compound_stmt:
    | function_def
    | if_stmt
    | class_def
    | with_stmt
    | for_stmt
    | try_stmt
    | while_stmt
    | match_stmt
...

factor:
    | '+' factor
    | '-' factor
    | '~' factor
    | power


...
```

docs.python.org/3/reference/grammar.html#full-grammar-specification

```
# PEG grammar for Python

...

simple_stmt[stmt_ty] (memo):
   | assignment
   | e=star_expressions { _PyAST_Expr(e, EXTRA) }
   | &'return' return_stmt
   | &('import' | 'from') import_stmt
...
compound_stmt[stmt_ty]:
   | &('def' | '@' | ASYNC) function_def
   | &'if' if_stmt
   | &('class' | '@') class_def
   | &('with' | ASYNC) with_stmt
   | &('for' | ASYNC) for_stmt
   | &'try' try_stmt
   | &'while' while_stmt
   | match_stmt
...

factor[expr_ty] (memo):
   | '+' a=factor { _PyAST_UnaryOp(UAdd, a, EXTRA) }
   | '-' a=factor { _PyAST_UnaryOp(USub, a, EXTRA) }
   | '~' a=factor { _PyAST_UnaryOp(Invert, a, EXTRA) }
   | power



...
```

python.gram

```python
def saberi():
      a = 5
      b = 3
      a = a + b
      print(a)
```

```python
import test
import ast
import inspect
tree = ast.parse(inspect.getsource(test.saberi))
ast.dump(tree)
```

```
Module(body=[FunctionDef(name='saberi', args=arguments(args=[],
vararg=None, kwonlyargs=[], kw_defaults=[], kwarg=None,
defaults=[]), body=[Assign(targets=[Name(id='a', ctx=Store())],
value=Num(n=5)), Assign(targets=[Name(id='b', ctx=Store())],
value=Num(n=3)), Assign(targets=[Name(id='a', ctx=Store())],
value=BinOp(left=Name(id='a', ctx=Load()), op=Add(),
right=Name(id='b', ctx=Load()))),
Expr(value=Call(func=Name(id='print', ctx=Load()),
args=[Name(id='a', ctx=Load())], keywords=[]))],
decorator_list=[], returns=None)])
```

pogledati: python-ast-explorer.com

```python
#!/usr/bin/python3
import ast
from fractions import Fraction

class IntegerWrapper(ast.NodeTransformer):
   """Wraps all integers in a call to Integer()"""
   def visit_Num(self, node):
       if isinstance(node.n, int):
           return ast.Call(func=ast.Name(id='Integer', ctx=ast.Load()),
                           args=[node], keywords=[])
       return node

class Integer(object):
   def __init__(self, value):
       self.value = value

   def __truediv__(self, other):
       if isinstance(other, Integer):
           return Fraction(numerator=self.value, denominator=other.value)

code = "print((1/10)+(2/10))"
print(code)
print()

print("Without AST transformation:")
exec(code)
print()

print("With AST transformation:")
tree = ast.parse(code)
tree = IntegerWrapper().visit(tree)
# Add lineno & col_offset to the nodes we created
ast.fix_missing_locations(tree)
co = compile(tree, "<ast>", "exec")
exec(co)
```

```
print((1/10)+(2/10))

Without AST transformation:
0.30000000000000004

With AST transformation:
3/10
```

greentreesnakes
wrap_integers.py

Izvršavanje

```
...

    DISPATCH();

    {
    /* Start instructions */
#if !USE_COMPUTED_GOTOS
    dispatch_opcode:
        switch (opcode)
#endif
        {

#include "generated_cases.c.h"



...
```

ceval.c

```
        case _LOAD_CONST: {
            PyObject *value;
            oparg = CURRENT_OPARG();
            value = GETITEM(FRAME_CO_CONSTS, oparg);
            Py_INCREF(value);
            stack_pointer[0] = value;
            stack_pointer += 1;
            break;
        }

        case _STORE_FAST: {
            PyObject *value;
            oparg = CURRENT_OPARG();
            value = stack_pointer[-1];
            SETLOCAL(oparg, value);
            stack_pointer += -1;
            break;
        }

        case _POP_TOP: {
            PyObject *value;
            value = stack_pointer[-1];
            Py_DECREF(value);
            stack_pointer += -1;
            break;
        }

        case _PUSH_NULL: {
```

executor_cases.c.h

```
/* Computed GOTOs, or
       the-optimization-commonly-but-improperly-known-as-"threaded code"
   using gcc's labels-as-values extension
   (http://gcc.gnu.org/onlinedocs/gcc/Labels-as-Values.html).

   The traditional bytecode evaluation loop uses a "switch" statement, which
   decent compilers will optimize as a single indirect branch instruction
   combined with a lookup table of jump addresses. However, since the
   indirect jump instruction is shared by all opcodes, the CPU will have a
   hard time making the right prediction for where to jump next (actually,
   it will be always wrong except in the uncommon case of a sequence of
   several identical opcodes).

   "Threaded code" in contrast, uses an explicit jump table and an explicit
   indirect jump instruction at the end of each opcode. Since the jump
   instruction is at a different address for each opcode, the CPU will …

   At the time of this writing, the "threaded code" version is up to 15-20%
   faster than the normal "switch" version, depending on the compiler and the
   CPU architecture.

   ...

   NOTE: care must be taken that the compiler doesn't try to "optimize" the
   indirect jumps by sharing them between all opcodes. Such optimizations
   can be disabled on gcc by using the -fno-gcse flag (or possibly
   -fno-crossjumping).
*/
```

ceval_macros.h

Alternative Python Implementations

This site hosts the "traditional" implementation of Python (nicknamed CPython). A number of alternative implementations are available as well, namely

- IronPython (Python running on .NET)
- Jython (Python running on the Java Virtual Machine)
- PyPy (A fast python implementation with a JIT compiler)
- Stackless Python (Branch of CPython supporting microthreads)
- MicroPython (Python running on micro controllers)

Other parties have re-packaged CPython. These re-packagings often include more libraries or are specialized for a particular application:

- ActiveState ActivePython (commercial and community versions, including scientific computing modules)
- pythonxy (Scientific-oriented Python Distribution based on Qt and Spyder)
- winpython (WinPython is a portable scientific Python distribution for Windows)
- Conceptive Python SDK (targets business, desktop and database applications)
- Enthought Canopy (a commercial distribution for scientific computing)
- PyIMSL Studio (a commercial distribution for numerical analysis – free for non-commercial use)
- Anaconda Python (a full Python distribution for data management, analysis and visualization of large data sets)
- eGenix PyRun (a portable Python runtime, complete with stdlib, frozen into a single 3.5MB - 13MB executable file)

If you want to host and run Python in the cloud, these implementations may be right for you:

- PythonAnywhere (freemium hosted Python installation which lets you run Python in the browser, e.g. for tutorials, showcases, etc.)

Cpython - bez JIT (za sad)

: my[py]

mypy 1.8.0 documentation

Search

FIRST STEPS

Getting started

Type hints cheat sheet

Using mypy with an existing codebase

TYPE SYSTEM REFERENCE

Built-in types

Type inference and type annotations

Kinds of types

Class basics

Annotation issues at runtime

Type hints cheat sheet

This document is a quick cheat sheet showing how to use type annotations for various common types in Python.

Variables

Technically many of the type annotations shown below are redundant, since mypy can usually infer the type of a variable from its value. See Type inference and type annotations for more details.

```
# This is how you declare the type of a variable
age: int = 1

# You don't need to initialize a variable to annotate it
a: int  # Ok (no value at runtime until assigned)

# Doing so can be useful in conditional branches
child: bool
if age < 18:
    child = True
else:
    child = False
```

Home Blog Examples Docs GitHub Wiki Roadmap About Contact

mypy

Mypy is an optional static type checker for Python that aims to combine the benefits of dynamic (or "duck") typing and static typing. Mypy combines the expressive power and convenience of Python with a powerful type system and compile-time type checking. Mypy type checks standard Python programs; run them using any Python VM with basically no runtime overhead.

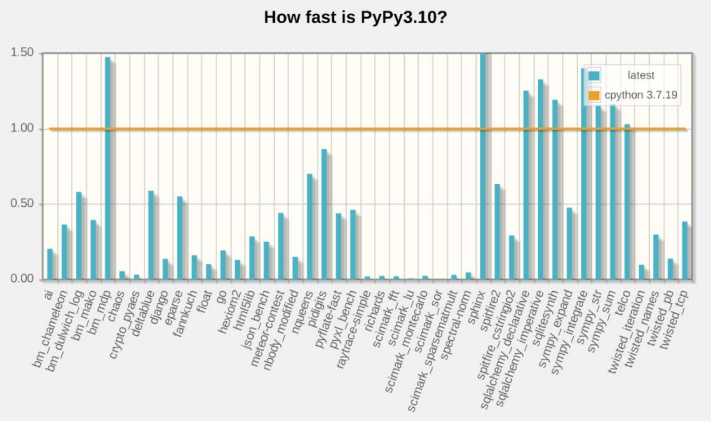
What's new

Mypy 1.8 released

21 Dec 2023: Mypy 1.8 was released. Read the blog post for the details. -Wesley Collin Wright

mypy.readthedocs.io
mypy-lang.org
peps.python.org/topic/typing

On average, PyPy is **about 3 times faster** than CPython 3.11. We currently support python 3.11 and 2.7.



PyPy (with JIT) benchmark times normalized to CPython. Smaller is better. Based on the geometric average of all benchmarks

PyPy
7.3

Search docs

- What is PyPy?
- Goals and Architecture Overview
- Downloading and Installing PyPy
- Building PyPy from Source
- Translating on Windows
- Frequently Asked Questions
- Differences between PyPy and CPython
- Writing extension modules for pypy
- Garbage collector documentation and configuration
- JIT hooks
- Resetting the JIT
- Application-level Stackless features
- The __pypy__ module
- PyPy's sandboxing features
- Software Transactional Memory
- Remote Debugging
- Contributing Guidelines
- PyPy's Configuration Handling
- Potential Project List
- Project Documentation

/ Welcome to PyPy's documentation! View page source

Welcome to PyPy's documentation!

Welcome to the documentation for PyPy, a fast, compliant alternative implementation of the Python language.

- If you want to find out more about what PyPy is, have a look at our What is PyPy? or consult the PyPy website.
- If you're interested in trying PyPy out, check out the installation instructions.
- If you want to help develop PyPy, please have a look at contributing and at the repo on https://github.com/pypy/pypy and get in touch (Contact)!

All of the documentation and source code is available under the MIT license, unless otherwise specified. Consult LICENSE.

Getting Started

- What is PyPy?
- Goals and Architecture Overview
- Downloading and Installing PyPy
- Building PyPy from Source
- Translating on Windows
- Frequently Asked Questions

Using PyPy

- Differences between PyPy and CPython

PyPy
7.3

Search docs

- Goals and Architecture Overview
- Frequently Asked Questions
- Cross-translating for ARM
- Cross-Translating for RISC-V
- Logging environment variables
- RPython Language
  - Definition

/ RPython Language View page source

RPython Language

Definition

RPython is a restricted subset of Python that is amenable to static analysis. Although there are additions to the language and some things might surprisingly work, this is a rough list of restrictions that should be considered. Note that there are tons of special cased restrictions that you'll encounter as you go. The exact definition is "RPython is everything that our translation toolchain can accept" :)

- CPython koristi optimizovani interpreter, ne JIT
  - quickening
  - zamena instrukcija drugim bytecode instrukcijama
  - dozvoljava dinamičku promenu koda
  - *stabilnost, praktičnost*
- PyPy koristi JIT
  - agresivno optimizuje
  - otežava debug
  - *performanse*

Realni jezici ne žive u idealnim teorijskim okvirima.

Dizajn jezika (specifikacija i prevodilac) je inženjerski posao.